

Predicting Profitable Customer Groups

End-to-end ML system: model, API, deployment and validation

Presented by: Venkata Nagendra Reddy Suda

19 August 2026

**Presenting to: Dr. Falko Trischler
Dr. Alexander Khachikyan**

Business question to production validation

The presentation follows the same path as the system: define the decision, prove the model, then show how it is engineered and validated.

01

Problem & data

Business decision, target definition and leakage boundary

Slides 3–4

02

ML evidence

Development-only selection, holdout lift and class-level errors

Slides 5–6

03

Engineering system

Architecture, toolchain, sequence, API, CI and deployment

Slides 7–12

04

Validation & close

Online ROI experiment and final recommendation

Slides 13–14

Recommend the best campaign action before spend

Each campaign round compares two customer groups. The system must choose an action before the profitable outcome is known.

Business decision

Contacting customers costs money. The recommendation must therefore be made before the campaign outcome and ROI are available.

- 1 Target Group 1
- 2 Target Group 2
- 0 Do not run the campaign

The assignment asks for

01

Understand historical outcomes

Measure how often Group 1, Group 2 or neither was profitable.

02

Predict the next action

Train a model using only information available before launch.

03

Deliver an operational system

Expose predictions through a user-facing frontend and backend APIs with tests and deployment architecture.

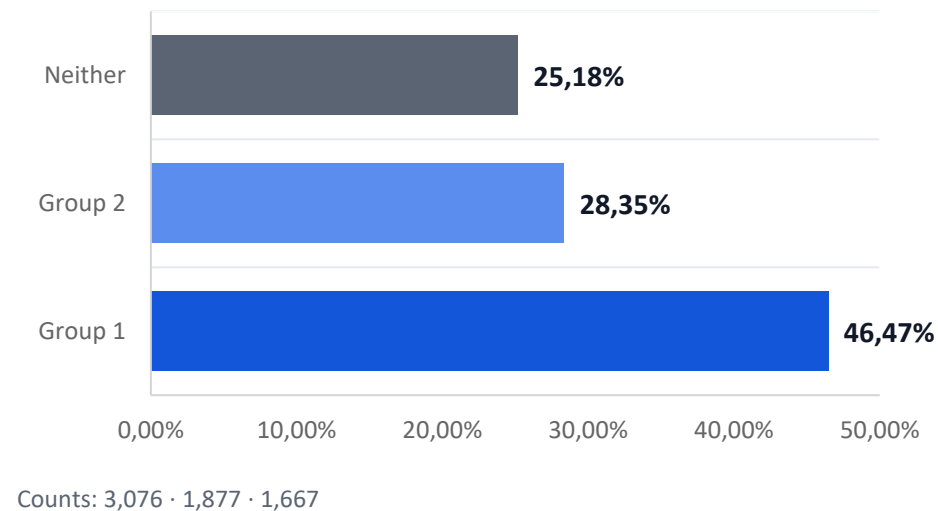
Decision-time boundary

Use only pre-campaign features. The Target and post-campaign fields are outcomes, not model inputs.

6,620 campaigns define the learning problem

The supplied Target already records the post-campaign outcome; the task is to predict it using only information available before launch.

Historical outcomes · 6,620 campaigns



67 inputs available before launch

20 Group 1 + 20 Group 2 + 27 comparison features

Mild class imbalance is preserved

Stratified splits · no SMOTE or synthetic rows
Random Forest uses balanced_subsample weights
Per-class recall and balanced accuracy remain visible

3 post-campaign fields excluded

g1_21 · g2_21 · c_28 — unavailable at decision time

DATA LIMITS

No date: future drift untested · No campaign/customer IDs: independence unverified · Anonymized features: predictive, not causal

Seven models compared with development-only CV

Random Forest won under the pre-set mean CV accuracy rule; the holdout did not influence model selection.

TRAIN · 4,236 · 64%

CALIBRATION
1,060 · 16%

HOLDOUT
1,324 · 20%

Selection protocol

- Stratified partitions
- Preprocessing fitted inside folds
- Mean CV accuracy is primary
- No holdout-informed override

Model evidence

Random Forest $57.81\% \pm 0.51$ pp

XGBoost $57.41\% \pm 0.90$ pp

Next five models: 52.29–56.56%

RF wins by 0.40 pp; separation is close.

Calibration boundary

The 1,060-row split is reserved for calibration and threshold decisions; it never trains Random Forest.

**Separate fit/evaluation halves were used.
Isotonic rejected: 6.1% ECE gain, below the pre-set 10% minimum.**

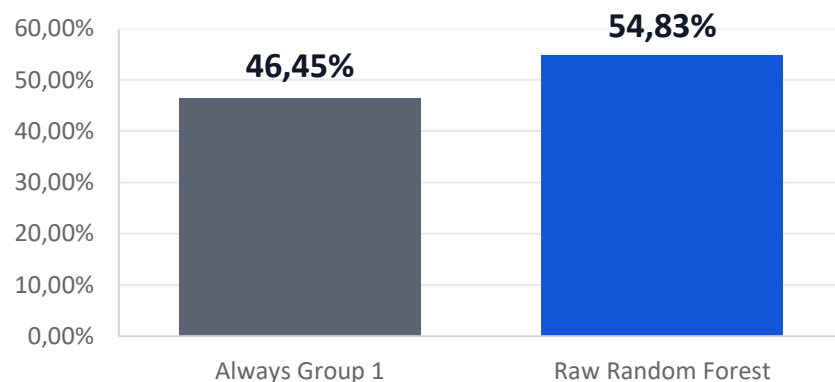
Evaluation boundary

Model selection is development-only. Because earlier EDA examined the full dataset, holdout results are confirmation—not pristine external validation.

Offline Lift is positive; class 0 remains weak

Held-out confirmation after exploratory reuse - not ROI and not fresh external validation.

Correct targeting decisions



+8.38 pp · 95% CI +5.66 to +11.18

Macro F1 44.82% · Balanced accuracy 47.80%

Actual ↓	Predicted class →		
	0 · Neither	1 · Group 1	2 · Group 2
0 · Neither	28	184	121
1 · Group 1	36	490	89
2 · Group 2	45	123	208

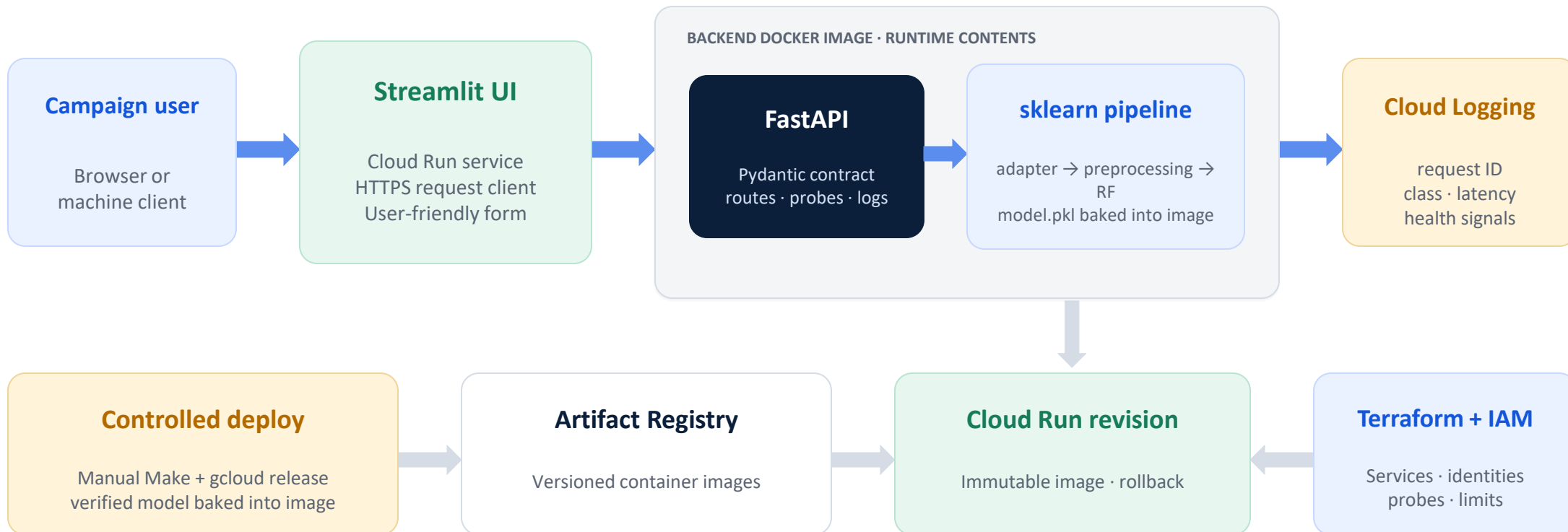
Recall · Class 0: 8.41% · Class 1: 79.67% · Class 2: 55.32%

305 of 333 decline campaigns were missed

Accuracy lift does not solve the 'neither profitable' decision. This is the primary model risk.

Browser to an immutable Cloud Run revision

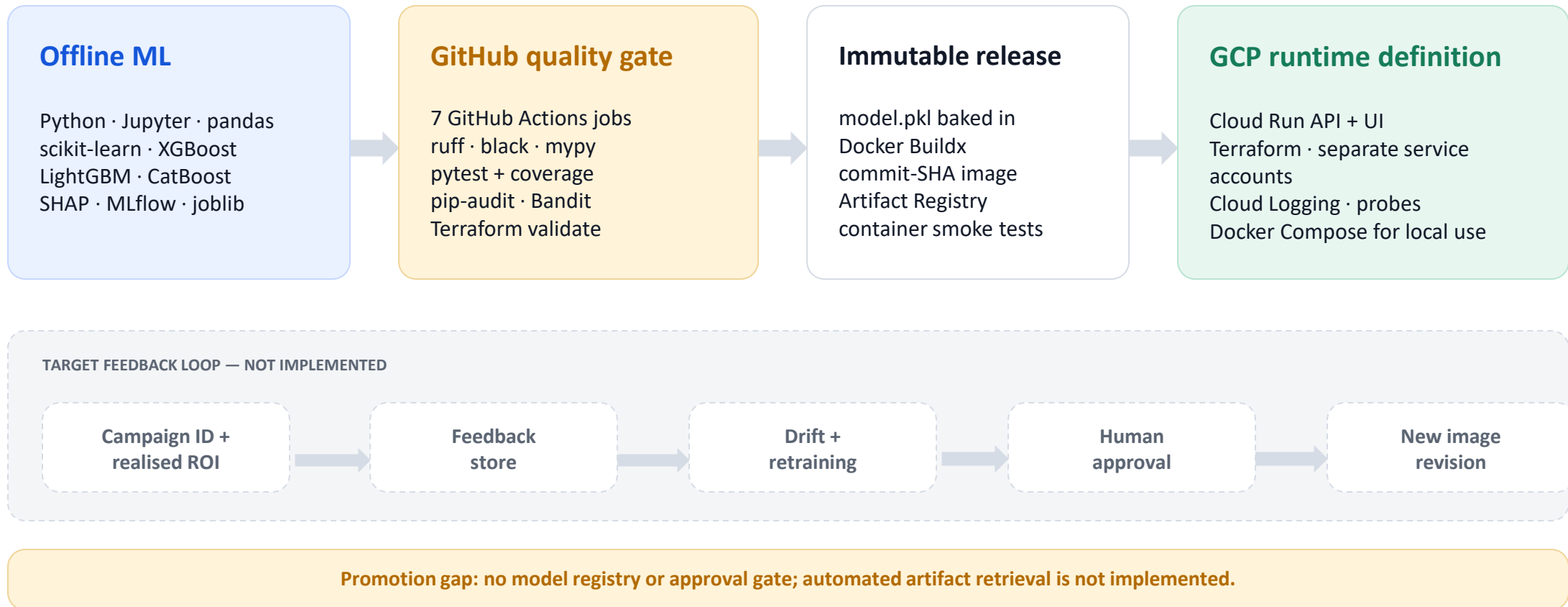
The deployed path is deliberately small: the API image contains the exact model artifact, and prediction responses include the model version.



Current boundary: model registry, approval gate, runtime drift alerts and automated artifact retrieval are not implemented.

Implemented today and proposed next

Current tooling is shown in solid boxes. The feedback and retraining loop is proposed and deliberately separated.



Tools map to clear lifecycle responsibilities

The design keeps experimentation optional, serving deterministic and infrastructure reproducible.

Typed API contract

FastAPI + Pydantic v2

Request validation, OpenAPI and dependency injection are native. Invalid fields fail before inference.

Immutable serving unit

Docker + Cloud Run

The model is baked into the image, so code and artifact roll back together. Trade-off: a 46.9 MiB model increases startup cost.

Cloud boundary as code

Terraform + IAM

Services, probes and identities are declared as code. The review endpoint remains public until frontend ID-token support exists.

Independent quality gate

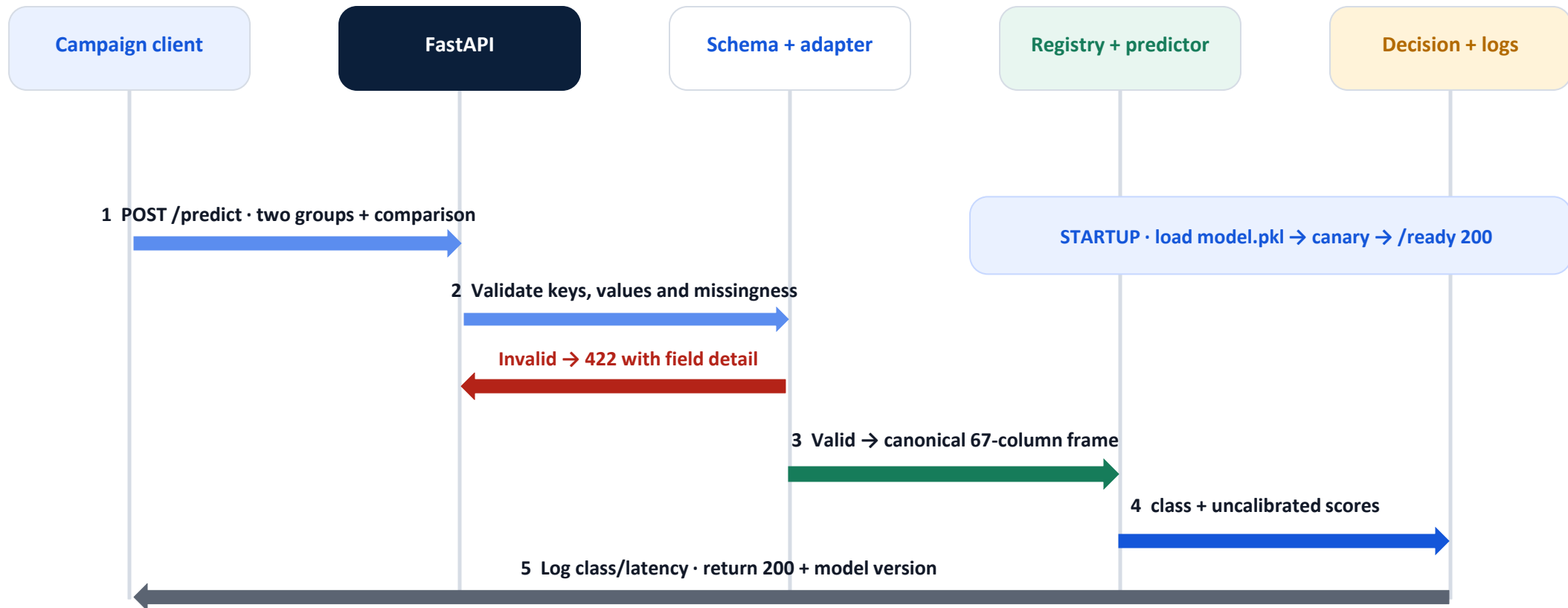
GitHub Actions

Seven jobs test data guards, code, security, Terraform and both container paths. Training and automatic deployment are intentionally absent.

Optional by design: MLflow tracks local experiments and SHAP may explain models; neither is required by the prediction service.

A request is validated, transformed, scored and logged

At startup, the model loads and passes a canary check; invalid requests fail before inference.



Decision Policy is optional and off by default. Returned confidence is a raw model score not ROI or a calibrated probability of profit.

Explicit contract, replaceable components

Typed request models and clear interfaces keep the HTTP layer separate from feature construction and inference.

Prediction API

POST	/predict	single recommendation
POST	/predict/batch	up to 1,000 rows
GET	/live · /ready · /health	runtime probes
GET	/model/info	model metadata
GET	/docs	OpenAPI interface

Component boundaries

FastAPI route + Depends

FeatureTransformer

BasePredictor
→ SklearnPredictor

Factory creates · Registry owns · interface can be replaced

Input and output policy

67 keys required · unknown/post-campaign fields rejected
null accepted up to 13 fields; NaN/∞ and |value| > 1e6 rejected
confidence = uncalibrated model score

Tests pass; runtime evidence is measured

The release evidence covers code quality, both container paths and a live sequential-request benchmark. Operational limitations remain explicit.

427

pytest cases

17 modules · unit + API + integration

91.43%

Measured code coverage

90% fail-under enforced

7

CI jobs defined

Push, PR and manual dispatch

0 / 50

failed live requests

Cloud Run sequential benchmark

GitHub Actions quality path

Data guard

release hygiene

Code + tests

lint · types · coverage

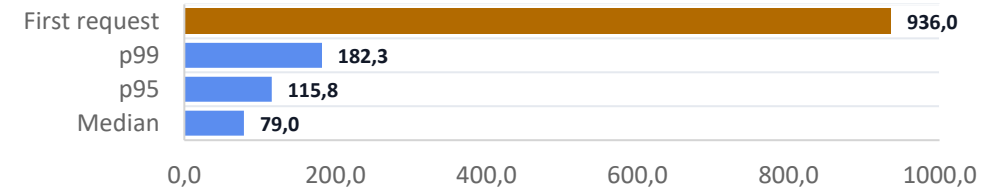
Security + image

audit · IaC · container ×2

Non-root images · synthetic fixtures · degraded health reported honestly

IMPLEMENTED /live · /ready · /health · Terraform · structured logs

Cloud Run latency profile · milliseconds

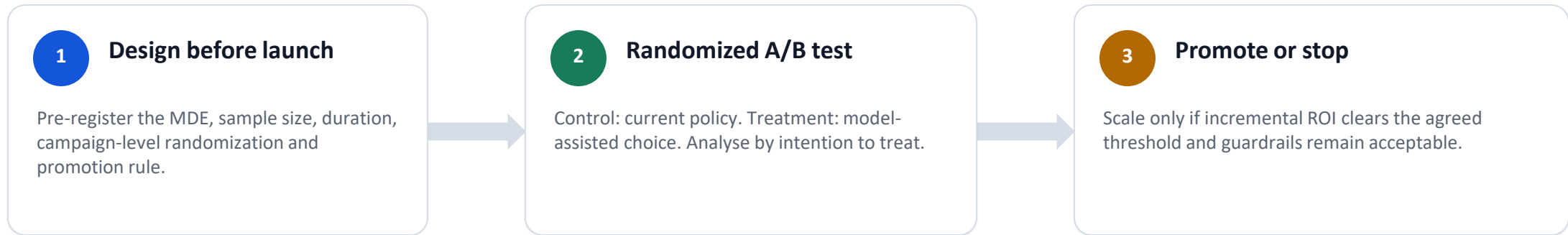


One ~15 s cold start was observed separately and could not be reproduced on demand.

PENDING auth · rate limiting · drift alerts · approval gate · artifact retrieval

Validate business impact with a controlled experiment

The offline metric is a screening signal. The production decision should be based on incremental ROI measured under random assignment.



Primary KPI	Secondary	Guardrails
ROI per campaign	Revenue · spend · skip rate · override rate	Latency · errors · subgroup harm

Decision rule: promote only when the pre-registered ROI criterion and operational guardrails both pass.

Proceed to a controlled pilot— not full automation

The engineering baseline is reproducible and offline lift is positive. Add access controls and operational monitoring before pilot traffic; weak decline recall blocks full automation.

1 Verify release

Run the full CI gate, redeploy the exact image, and capture API/UI evidence.

2 Secure pilot

Add authentication, rate limiting and live alerts; keep decisions reversible.

3 Validate impact

Run a powered campaign-level A/B test before claiming ROI improvement.

Thank you

Questions & discussion